Boolean masking

Previously, we investigated several different ways of selecting data in a dataframe, including column selection, row selection, and selection of combinations of rows and columns using name-based and integer-based indexing. In this video, you'll learn how to filter the data in the dataframe based on value-based conditions. You know that boolean is used to describe any binary variable with two possible values: true or false. With pandas, you can use a powerful technique known as Boolean masking. Boolean masking is a filtering technique that overlays a Boolean grid onto a dataframe in order to select only the values in the dataframe that align with the True values of the grid. Data professionals use boolean masking all the time, so it's important that you understand how it works. Here's an example.

Suppose you have a dataframe of planets, their radii, and the number of moons each planet has. Now, suppose that you want to keep the rows of any planets that have fewer than 20 moons and filter out the rest. A boolean mask is a panda Series object indicating whether this condition is true or false for each value in the "moons" column. The data contained in this series is type bool. Boolean masking effectively overlays this boolean series onto the dataframe's index. The result is that any rows in the dataframe that are indicated as True in the Boolean mask remain in the dataframe, and any row

that are indicated as False get filtered out. Here's how to perform this operation in pandas.

We'll begin by creating a dataframe from a predefined dictionary using the pandas DataFrame function. The dataframe is called "planets." The next step is to create the boolean mask by writing a logical statement. Remember, the objective is to keep planets that have fewer than 20 moons and to filter out the rest. So, we define the mask by writing "planets at the moons column is less than 20". This results in a Series object that consists of the row indices where each index contains a True or False value depending on whether that row satisfies the given condition. This is the boolean mask. To apply this mask to the dataframe, insert it into selector brackets and apply it to the dataframe. It's also possible to apply the conditional logic directly to the dataframe, skipping the part where we assign it to a variable named "mask." But breaking out the steps individually can make the code easier to follow. Note that we haven't permanently modified the dataframe. Applying the boolean mask using the conditional logic only gives a filtered "view" of it. So, when you call the planet's variable again it returns the full dataframe. However, we can assign the result to a named variable.

This may be useful if you'll need to reference the list of planets with moons under 20 again later. Sometimes you'll need to filter data based on multiple conditions. Pandas uses logical operators to indicate which data to keep and which to filter out and statements that use multiple conditions. These operators are: The ampersand for "and" the vertical bar for "or" and the tilde for "not". Let's review how this works. Here's how to create a boolean mask that selects all planets that have fewer than 10 moons or greater than 50 moons. Notice that each condition is self-contained and a set of parentheses,

and the two conditions are separated by a vertical bar, which is the logical operator that represents "or".

It's very important that each component of the logical statement be in the parenthesis. Otherwise, your statement will throw an error, or worse, return something that isn't what you intended. To apply the mask, call the dataframe and put the statement or the variable it's assigned to in selector brackets. Here's an example of how to select all planets that have more than 20 moons, but not planets with 80 moons and not planets with a radius less than 50,000 kilometers. Let's break it into pieces. First is a statement for planets. In parentheses, we put "planets at the moons column must be greeter than 20" and close the parenthesis.

Then we use the "and" "not" operators before parentheses "planets at the moons column equals 80," close parentheses. Then we again use the "and" "not" operators before parentheses "planets at the radius column less than 50,000," close parentheses. When we apply the mass to the dataframe, we're left with just one planet: Saturn, with 83 moons and a radius of 58,232 kilometers. There are a near infinite number of ways to select and filter data using the basic tools that you've learned so far. As always, it takes a lot of practice before you know exactly how to execute every selection statement that your work requires. So make sure to bookmark any and all resources that you find helpful to reference. Keep up the good work and I'll meet you in the next video.

Boolean masking

A filtering technique that overlays a Boolean grid onto a dataframe in order to select only the values in the dataframe that align with the True values of the grid